

Reading a looped model before it finishes: Jacobian lenses across recurrent depth

MATS 12.0 application · Jan Kirin · September 2026

Executive summary

Ouro-2.6B is a looped transformer: the same 48 layers run four times, so there are 192 places to read a hidden state. J-Lens is Anthropic's Jacobian lens, meant to recover a model's intermediate variables. Neel's list asks how much better it really is than the logit lens and why. I asked that on a looped model, plus the question I actually care about: if you read the model at loop 1 or 2, do you see what it will say at the end?

The result I would defend first: on multihop prompts with a known intermediate, the plain logit lens beats the Jacobian lens at every loop before the last. On 90 multihop items, excess hit@10 over matched controls for the Jacobian lens is +0.003, -0.025, +0.084, +0.451 across loops 1 to 4. The logit lens gets +0.358, +0.458, +0.500, +0.483. The paired difference is below zero at loops 1 to 3 (95% intervals [-0.46, -0.25], [-0.58, -0.38], [-0.52, -0.31]) and unresolved at loop 4 ([-0.11, +0.04]). On 51 arithmetic items the Jacobian lens loses at loop 1 ([-0.35, -0.14]) and nothing is resolved after that. It is not blind. Arithmetic candidate-set top-1 is 0.608 at loop 1. It is much worse than a free baseline on the early multihop loops.

The monitoring part went the opposite way from what I wrote down beforehand. I fitted a lens to each loop's own exit and compared it with the lens that reads the final exit. I expected the gap between them to shrink with depth. At the primary readout position it does not. Mean KL from local to eventual readout is 4.9, 4.6 and 6.5 nats at loops 1 to 3, and last-eight-layer top-1 agreement is 0.002 to 0.018. One token earlier, KL declines, but top-1 agreement still stays at or below 0.005: the trend is position-sensitive.

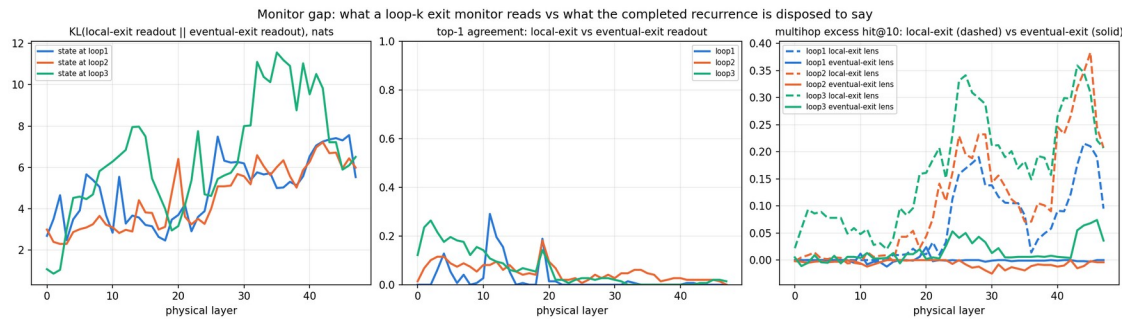


Figure 2. What a loop-k monitor reads versus what the finished recurrence says. Left: $KL(\text{local-exit readout} || \text{eventual-exit readout})$ by layer for states at loops 1 to 3. Middle: top-1 agreement between the two readouts. Right: multihop excess hit@10, local-exit lens (dashed) versus eventual-exit lens (solid).

Which loop the lens was fitted at explains 98 to 99 percent of the variance in the multihop readout. Which loop produced the state explains 1 percent. The lens carries the loop it was fitted to, not the state you hand it. I am calling that a description, not an explanation.

The simplest story: Ouro trains every loop's exit through the same output head, so the logit lens is trained into every loop for free, while a Jacobian from loop k to the final exit averages over routes that later loops rewrite. I have not tested this against a looped model without that training, so it stays a hypothesis.

What I checked. The recorder reproduces the stock forward pass bit for bit (18 of 18 comparisons), locally and on the B300. Review caught a control statistic that inflated every excess (max of means against mean of maxima); fixed, all six evaluations re-derived. Also fixed: a probe split leaking mirror pairs, and a matcher counting "11" as correct for "1". Two claims retracted after audit. The eight intervals are unadjusted item bootstraps. A supervised probe baseline is in section 5.

Limits, short version: one model, one estimator, fits of at most 80 prompts locally, and the intermediates are token labels, not proven causal steps. Next: a tuned lens baseline, a matched 2x2 estimator control, and a looped model without shared-head supervision.

On a B300, 100-prompt lenses reproduce this: multihop differences -0.36, -0.48, -0.41, -0.02, a flat fit-size ladder, and the same local-versus-eventual gap (KL 4.8, 4.1, 6.3 nats). The pod's final-exit and fit-size analyses passed strict provenance checks. Its all-exit evaluations finished but timed out before publication, so I reran them locally over the retained B300-fitted lenses.

Randomly selected examples

Five items drawn with `random.Random(0).sample` from the 142 items with a scorable, non-leaked intermediate (of 148). Not cherry-picked. Rank is over the whole vocabulary, best over the 48 layers in each loop; 0 means top-1, under 10 means hit@10.

item	prompt	intermediate, target	model said	Jacobian lens best rank, loops 1-4	logit lens best rank, loops 1-4
atomic-26-symbol	Fact: The chemical symbol for the element with atomic number 26 is	iron -> Fe	Fe. (correct)	979 / 1716 / 222 / 0	0 / 0 / 0 / 0
rhyme-hive-plusone	Fact: One more than the number whose name rhymes with hive is	five -> 6	100. (wrong)	3 / 53 / 7 / 2	4 / 4 / 3 / 6
add-sub-left-right	$10 + 5 - 3 =$	15 -> 12	12 (correct)	39 / 1 / 16 / 3	2 / 9 / 23 / 4
nested-add-mult-sub	$((1 + 2) * 3) - 4 =$	9 -> 5	9 (wrong)	29 / 17 / 0 / 0	0 / 0 / 0 / 0
word-sub-mult	ten minus two times three equals	6 -> four	Answer: (wrong)	179 / 1725 / 213 / 12	8 / 4 / 6 / 11

Two things worth seeing in the raw rows. On atomic-26-symbol the Jacobian lens applied to loop 1 ranks "iron" 979th and its top three are the numbers 11, 14 and 12: the final-exit lens on an early state reads the wrong domain entirely, then snaps to rank 0 at loop 4. On nested-add-mult-sub the model's wrong answer is the intermediate itself, so both lenses "reading 9" are reading the output. And on word-sub-mult the model never answers at all. That is what the 72 of 148 correctness rate looks like item by item.

1. Setup and checks

Model: ByteDance/Ouro-2.6B, revision 1ed04250, four passes over 48 shared layers; a location is $\text{loop} * 48 + \text{layer}$, loop 1 first. Lens code: the released J-Lens estimator, revision 581d3986, unchanged. A hook on each physical layer fires only on the recurrent step I ask for and stores the residual stream there. Before fitting anything I checked that hooked and plain forwards agree exactly, that each recurrent exit equals the model's own forward at that step, that loop-1 layer-0 input equals the embeddings and the four visits have distinct storage and values, that gradients through the four visits of a layer are distinct

and causal in position, and that a stock-consistency run passes. All 18 required comparisons are equal, not merely close, on my laptop and again on the B300.

The lens: for source location l and a target exit, $\text{lens}_l(h) = \text{unembed}(j_l h)$, with j_l the average over prompts and valid positions of the Jacobian of the target state with respect to h . One forward per prompt with the prompt replicated dim_batch times, then $\text{ceil}(d_{\text{model}} / \text{dim_batch})$ backward passes with one-hot cotangents at every valid target position.

2. Stimuli and metric

The released J-Lens evaluation sets: 93 multihop items with a named intermediate (90 after dropping items whose intermediate appears in the prompt; 100 clean slots) and 55 order-of-operations items with a numeric intermediate (51 clean). Wikitext prompts for fitting. At the token before the target, for every layer in a loop, hit@10 is whether the intermediate is in the lens's top ten. The primary statistic is the item-level difference between the best hit@10 over layers for the true intermediate and the same statistic averaged over same-kind control names. Slots are averaged within item first. Intervals are paired item bootstraps, 20,000 draws, fixed seed. Correctness is a boundary-aware match ("11" is not "1"): 72 of 148 stimuli are model-correct. Before any lens, each loop's own top-1 agrees with the final exit 0.409, 0.656, 0.785, 1.000 of the time on multihop and 0.527, 0.855, 0.964, 1.000 on order-of-operations: early loops commit to a different token about half the time.

3. Jacobian lens against the logit lens

Any-layer excess hit@10 over matched same-kind controls:

population	readout	loop 1	loop 2	loop 3	loop 4
multihop (90 items)	Jacobian lens	+0.003	-0.025	+0.084	+0.451
	logit lens	+0.358	+0.458	+0.500	+0.483
	Jacobian minus logit	-0.355	-0.483	-0.416	-0.032
order-ops numeric (51 items)	Jacobian lens	+0.054	+0.273	+0.181	+0.193
	logit lens	+0.306	+0.152	+0.131	+0.213
	Jacobian minus logit	-0.252	+0.121	+0.050	-0.020

Paired 95% intervals for Jacobian minus logit, unadjusted: multihop [-0.460, -0.250], [-0.583, -0.382], [-0.519, -0.313], [-0.105, +0.039]; arithmetic [-0.354, -0.136], [-0.005, +0.246], [-0.081, +0.186], [-0.069, +0.023]. Same shape on both tasks: the logit lens reads the intermediate from loop 1, the Jacobian lens fitted to the final exit reads almost nothing on multihop until loop 4 and never beats the logit lens on arithmetic. Rerun on the B300 with 100-prompt lenses the paired multihop differences are -0.356 [-0.463, -0.249], -0.479 [-0.579, -0.377], -0.407 [-0.510, -0.304], -0.022 [-0.096, +0.056]; arithmetic -0.255 [-0.362, -0.139], +0.140 [+0.020, +0.264], +0.089 [-0.045, +0.226], -0.021 [-0.072, +0.024], so loop 2 on arithmetic now resolves in the Jacobian lens's favour.

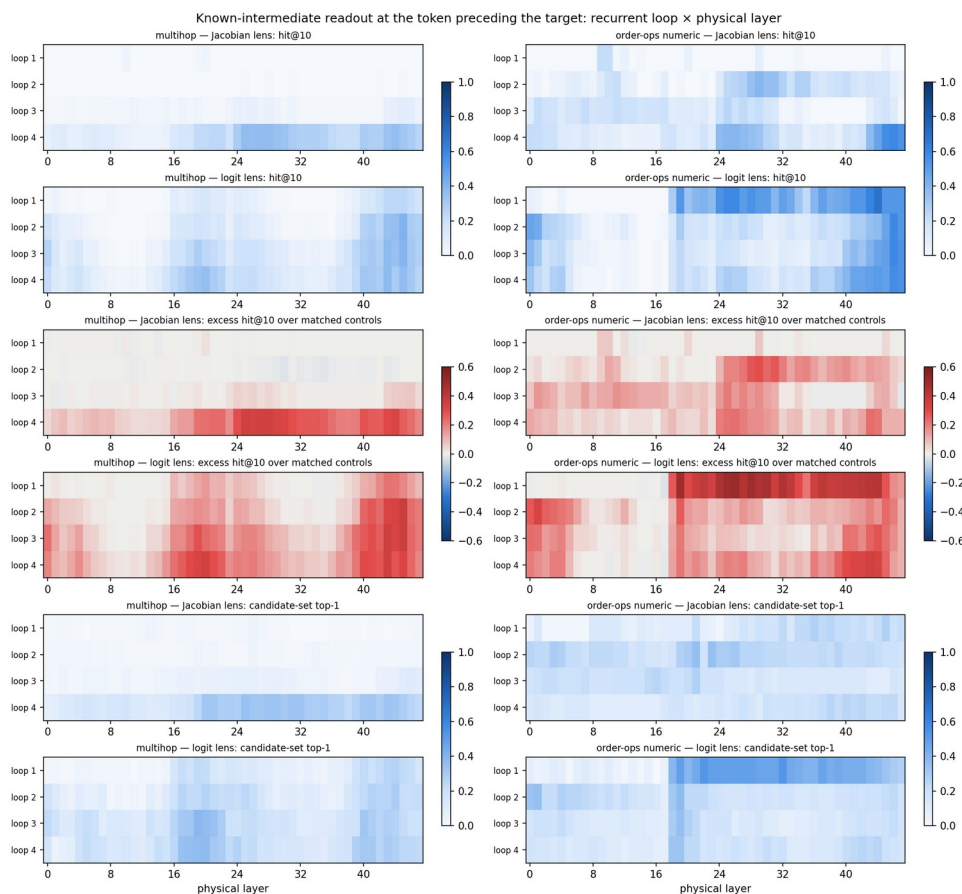


Figure 1. Known-intermediate readout at the token before the target, recurrent loop (rows) by physical layer (columns). Top: hit@10 for the Jacobian lens and the logit lens. Middle: excess hit@10 over matched same-kind controls. Bottom: candidate-set top-1. Left multihop, right order-of-operations.

4. Local against eventual exit, and cross-loop application

Lenses fitted to loop k's own exit (24 prompts each) against the final-exit lens (32 prompts); loop 4 is equal by definition.

loop	mean KL local vs eventual	top-1 agreement, last 8 layers	local minus eventual excess
1	4.903	0.002	+0.065
2	4.567	0.018	+0.114
3	6.526	0.008	+0.151

I had written down that the first column should decrease. At this readout position it does not. With the B300 N=100 lenses for all four exits (evaluation run locally on my GPU from the pod-fitted lenses) the rows are 4.771 / 0.003 / +0.076, 4.098 / 0.033 / +0.114 and 6.300 / 0.014 / +0.159. One token earlier KL does decrease, 5.239, 4.631, 3.911, while top-1 agreement remains at most 0.005; the KL trend is position-sensitive. Applying a lens fitted at loop i to states from loop j, the multihop readout is set by i: fit-loop share of variance 0.98 at the primary position and 0.96 one token earlier, with state-loop share 0.01 at both. Replicated association; "remaining horizon" and estimation difficulty are both live readings and I cannot separate them with what I have.

5. Probe baseline and fit size

Arithmetic family $(a + b) * c =$, 648 prompts in 45 unordered (a, b) clusters, 17 possible intermediates. Five folds keep mirror pairs together; 72 held-out prompts have a label absent from their training fold, so the fair set is 576 prompts in 39 clusters. For each fold and method the layer is chosen on the other folds. Chance 0.059, majority baseline 0.125.

readout	loop 1	loop 2	loop 3	loop 4
supervised probe	0.594	0.559	0.356	0.248
logit lens	0.764	0.500	0.179	0.311
Jacobian lens	0.392	0.500	0.148	0.120

Jacobian minus logit is below zero at loop 1 ([-0.548, -0.221]) and loop 4 ([-0.284, -0.105]); probe minus logit is below zero at loop 1 and above zero at loop 3 ([+0.061, +0.342]). Pointwise intervals, no family-wide claim. Nested fits at 8, 32, 56 and 80 prompts leave multihop loop-1 excess near zero and loop 4 near 0.45; more fitting data does not recover the early-loop readout, and nested fits say nothing about run-to-run variance. On the B300 the ladder extends to 100 prompts with the same shape: multihop loop-1 excess +0.007, +0.018, -0.007, -0.012, +0.002 and loop-4 excess +0.462, +0.464, +0.451, +0.451, +0.472 for 8, 32, 56, 80, 100 prompts.

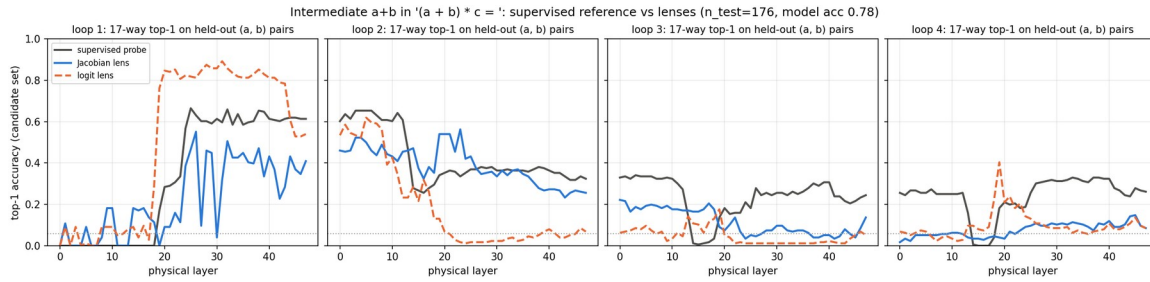


Figure 3. Arithmetic intermediate $a+b$ in $(a + b) * c =$: 17-way top-1 on held-out (a, b) pairs by physical layer, one panel per loop. Supervised probe (black), Jacobian lens (blue), logit lens (orange, dashed). Dotted line is chance (0.059).

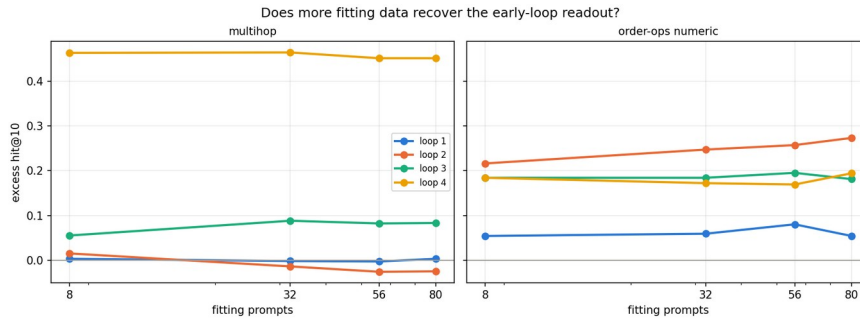


Figure 4. Does more fitting data recover the early-loop readout? Excess hit@10 by number of fitting prompts (8, 32, 56, 80; nested prefixes), one line per loop. Left multihop, right order-of-operations.

6. What I checked, in order

1. Recorder equality, before a single lens was fit.
2. Control aggregation: the first version compared mean-over-layers of the control names with max-over-layers of the true name. Every excess shrank once both used max. All six evaluation directories regenerated.
3. Probe leakage: ordered pairs leaked (a, b) and (b, a) across folds. Unordered clusters fixed it and cost 72 prompts.
4. Correctness matcher: prefix matching gave three false positives; boundary-aware matching removed them.
5. Overclaims: "never beats the logit lens" became "never significantly beats"; two values reported as single-prompt norms were modeled scatter with two invalid moments silently zeroed, now reported as modeled moments with the invalid count.
6. Three independent agent reviewers plus my own re-derivation read the analysis code. One found the control bug; one refuted a claim of mine that the probe was not data-limited.

7. Limits and next

- One model. The effect may be specific to per-loop shared-head training.
- Fits of at most 80 prompts locally, nested, no fixed-size replicates.
- The B300 rerun: 100-prompt fits for all four exits completed and all 17 fit shards were published with hash-verified receipts. The pod's exit-3 and fit-size evaluations passed strict provenance validation and analysis, replicating the local numbers within a few thousandths (multihop paired difference -0.356, -0.479, -0.407, -0.022). Both pod-side all-exit evaluations completed, but the hard timeout arrived during analysis before those trees were published. I therefore reran them locally on the 5070 Ti over the retained B300-fitted lenses; strict lineage validation and analysis pass. The result reproduces the local-exit finding: mean KL 4.8, 4.1, 6.3 nats at loops 1 to 3, top-1 agreement at most 0.033. The canonical report and paid-verifier stages did not complete on the pod.
- Eight pointwise intervals; unresolved is not equal.
- Intermediates are task-defined tokens; causal use is not shown.
- The local-versus-eventual KL depth trend changes one token earlier, although top-1 agreement remains near zero; conclusions about convergence are position-specific.
- The 2x2 estimator control (position-summed against per-position, mean against alternatives) was not run, so "estimator-specific" is a hypothesis.
- Also, a confession. The plan was simple: rent a B300 on the 4th, let it fit overnight, wake up to results, write everything up a full day before the deadline. So I went to bed and left Claude Opus to oversee the run until morning. Both I and the session went to sleep. The pod did not. About \$52 later there was nothing to retrieve. So the controller got rebuilt to kill the pod on any failure, and then the rerun took nine launches. Nine. Eight pods died inside seventy seconds, each on something a dry run on my laptop could not have seen: RunPod leaves the GPU name out of its response, sshd drops the token, the image has no hf CLI, pip wants --break-system-packages, and so on down the list. The ninth finally ran and managed one prompt every 29 minutes. PyTorch had given each fit process 192 threads on a 192-core box and was spending its life in OpenMP. Capped at 8 it went 43x faster. The B300 and I are on speaking terms now. Barely.

Next: a tuned lens baseline, the matched 2x2 estimator control, fixed-size replicate fits, and a looped model without shared-head supervision (or an Ouro ablation) to test the mechanism.

8. Time and links

I spent approximately 15 active hours on the project, counting project reading, recorder and validation work, estimator integration, analysis and the write-up. I excluded rental setup, waiting on fits and filling out the form.

Honestly, this write-up got done in the night before the deadline, with the B300 still fitting in the background and me watching the receipts land one at a time. Not recommended. Not the plan either. See the confession above.

Code: <https://github.com/VykosMolt/Hidden-State-Evaluator>. Results and receipts: [Vykos/ouro-jlens-results](#), run `jlens-b300-20260905-0359`.